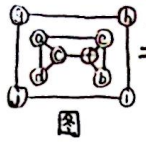
十大排序算法

方法	$C_{avg}(n)$	$C_{worst}(n)$	空间复杂度	稳定性
插入px	$O(n^2)$	$O(n^2)$	$O(1)$	✓
希尔px	$O(n^{1.3})$	$O(n^2)$	$O(1)$	✗
选择px	$O(n^2)$	$O(n^2)$	$O(1)$	✗
堆px	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	✗
冒泡px	$O(n^2)$	$O(n^2)$	$O(1)$	✓
快速px	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	✗
归并px	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	✓
以上为比较型排序，以下为非比较型排序				
计数px	$O(n+k)$	$O(n+k)$	$O(n+k)$	✓
桶px	$O(n+k)$	$O(n^2)$	$O(n+k)$	✓
基数px	$O(n*k)$	$O(n*k)$	$O(n+k)$	✓

DFS遍历



图

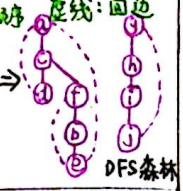
$a_{1,6}$

↳入栈顺序 ↳出栈顺序

$\Rightarrow$ $d_{2,1}$ $c_{3,5}$ $a_{4,6}$ $e_{6,2}$ $b_{5,3}$ $j_{10,7}$ $f_{9,4}$ $i_{9,8}$ $h_{8,9}$ $g_{7,10}$

遍历栈

实线：树枝边
虚线：回边



DFS森林

BFS遍历

a_1

↳入队(或出队)顺序

图同上 $\Rightarrow$ a_1 c_2 d_3 e_4 f_5 b_6 g_7 h_8 j_9 i_{10}

遍历队列

实线：树枝边
虚线：交叉边

BFS森林

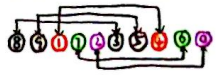
	DFS	BFS
数据结构	栈(stack)	队列(queue)
邻接链表的效率	$\Theta(\lvert V\rvert+\lvert E\rvert)$	$\Theta(\lvert V\rvert+\lvert E\rvert)$
邻接矩阵的效率	$\Theta(\lvert V\rvert^2)$	$\Theta(\lvert V\rvert^2)$
应用	都可以判断是否有环、是否连通；DFS求关节点；BFS求最短路径	

希尔排序

输入数组 ⑧⑦①②③⑥⑤④⑨

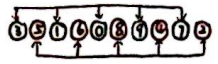
⇓

第1轮分组 增量d=5



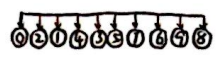
⇓

第2轮分组 增量d=2



⇓

第3轮分组 增量d=1



⇓

输出数组 ①②③④⑤⑥⑦⑧⑨

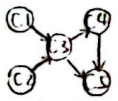
多轮分组插入排序 → 在已有序的A[0:n-2]中插入A[n-1]（从后往前扫描）

- 分组方式取决于增量序列
- 优先对距离较远的元素进行大小比较

$C_{avg}(n)=O(n^{1.3})$

拓扑排序 ⇒ 无环有向图才有解

① 基于DFS



有向图

$\Rightarrow$ $C5_{0,1}$ $C4_{1,2}$ $C3_{2,3}$ $C1_{1,4}$ $C2_{5,5}$

DFS遍历栈

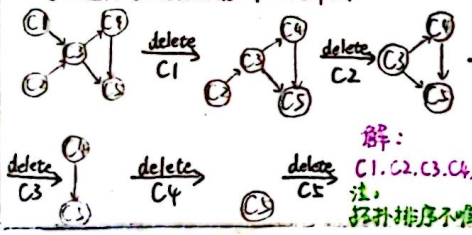
$\Rightarrow$ 出栈次序：C5, C4, C3, C1, C2

拓扑排序表：倒置

C2 C1 → C3 → C4 → C5

② 减(-)治

每次迭代时删除没有input的节点



解：C1, C2, C3, C4, C5

注：拓扑排序不唯一

折半查找

$\Theta(\log n)$

例：K=70

下标	0	1	2	3	4	5	6	7	8	9	10
值	3	14	27	31	39	42	55	70	74	81	85
迭代1	l					m					r
迭代2							l	m			r
迭代3							l,m	r			
迭代4								l,m,r			

约瑟夫斯问题

求L(n,m) ⇒ n人圆圈，数到m淘汰，最终幸存的序号

m=2时，$\begin{cases} n=2k, & J(2k)=2J(k)-1 \\ n=2k+1, & J(2k+1)=2J(k)+1 \end{cases}$

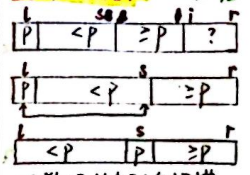
递推公式：

$L(k+1,m)=[L(k,m)+m] \bmod (k+1)$

$L(1,m)=1$

Lomuto划分

以第一个元素对数组划分



若A[i]<p 则交换A[i],A[s]

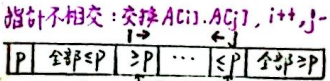
指针i搜索完后，交换中轴

完成划分

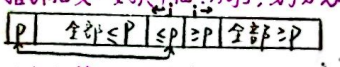
对数组从左到右扫描

Hoare划分

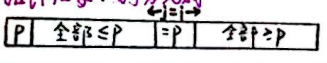
指针不相交：交换A[i],A[j]，i++, j--，继续扫描



指针相交：交换中轴，A[j]，划分完成



指针相等：划分完成



快速选择算法

$C_{best}(n)=\Theta(n)$ $C_{worst}(n)=\Theta(n^2)$ $C_{avg}(n)=\Theta(n)$

① 基于Lomuto划分，得到分割位置s

② 若s=k-1，则中轴p就是第k小的元素

③ 若s>k-1，则数组第k小元素为左边第k小元素，在左边找

④ 若s<k-1，则数组第k小元素为右边第(k-s)小元素，在右边找

合并排序

先对数组依次一分为二，直至不可分割，再自底向上依次递归排序并做合并

⇒ 关键在合并

快速排序

运用划分算法(Lomuto/Hoare/…)将A[0:n-1]分成3段：≤A[s], A[s], ≥A[s]，递归调用快排对左右2段分别排序，最后合并 ⇒ 关键在划分

二叉查找树的查找和插入

$C_{worst}(n)=\Theta(n)$ $C_{avg}(n)=1.39\log_2 n$

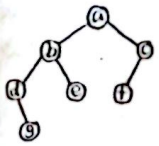
类似，先判断在根/左子树/右子树，然后决定是否进入左子树/右子树迭代查找

效率通用递推式

$T(n)=aT(n/b)+f(n)$，其中 $f(n)\in\Theta(n^d)$，$n\geq 0$

主定理 $\begin{cases} 当\ a<b^d,\ T(n)\in\Theta(n^d) \\ 当\ a=b^d,\ T(n)\in\Theta(n^d\log n) \\ 当\ a>b^d,\ T(n)\in\Theta(n^{\log_b a}) \end{cases}$

二叉树遍历



前序preorder：根→左子树→右子树
a.b.d.g.e.c.f

中序inorder：左子树→根→右子树
d.g.b.e.a.f.c

后序postorder：左子树→右子树→根
g.d.e.b.f.c.a

平衡查找树(AVL树)

查找、删除、插入均为$\Theta(\log n)$

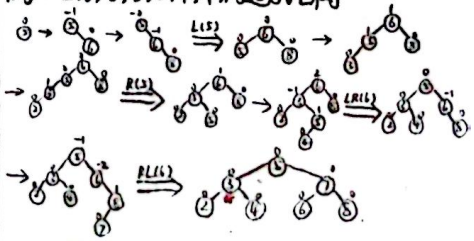
平衡因子：该节点左右子树高度差 ⇒ AVL：平衡因子均为0/+1/-1

LL插入 ⇒ 左单转　RR插入 ⇒ 左单转

LR插入 ⇒ 左右双转　RL插入 ⇒ 右左双转

[左右双转]

例：5,6,8,3,2,4,7 构造AVL树



平衡查找树(2-3树)

查找、删除、插入均为$\Theta(\log n)$

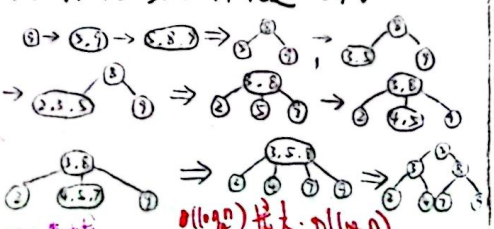
包含n个节点的2-3树的高度 $h:\log_3(n+1)-1\leq h\leq\log_2(n+1)-1$

(1) 插入

① 键从插入2节点 ⇒ 结构不变换

② 键从插入3节点 ⇒ 时分裂成2个节点，min放入第1个叶子，max放入第2个叶子，中间值升到原来的父母节点

例：9,5,8,3,2,4,7 构造2-3树



(2) 查找

① 若是2节点，1次比较后，进入其中一个子树继续查找

② 若是3节点，最多2次比较后，进入其中一个子树继续查找

$\Theta(\log_2 n)$ 最大：$O(\log_2 n)$

堆排序

堆：基本完备；父母>孩子

堆的根为最大元素；可用数组从上到下、从左到右记录堆元素

(1) 堆的构造

① 自顶向下(键的连续插入)

把新节点上附加在最后一个叶子后，然后与其父母键比较，上移至适处

② 自底向上 $C_{worst}(n)\in O(n)$

按从下向上、从右至左顺序比较大小，进行交换以满足性质

例：2,9,7,6,5,8 自底向上构造堆

(2) 最大键删除

根键与最后一个键交换 → 删除最大键 → 自底向上构造新堆

(3) 堆排序 $O(n\log n)$

Step1：自底向上构造堆 $O(n)$

Step2：逐个删除最大键 $O(n\log n)$

Step1：

2 9 7 6 5 8
2 9 8 6 5 7
2 9 8 6 5 7
9 2 8 6 5 7
9 6 8 2 5 7

Step2：

9 6 8 2 5 7
7 6 8 2 5 | 9
8 6 7 2 5
5 6 7 2 | 8
7 6 5 2
2 6 5 | 7
6 2 5
5 2 | 6
5 2
2 | 5

worst case：

基于比较的排序 ⇒ $\geq\lceil\log_2 n!\rceil\approx n\log_2 n$ 次比较

基于比较的查找 ⇒ $\geq\lceil\log_2(n+1)\rceil$ 次比较

扫描全能王 创建

字符串匹配 Horspool 算法 $\Theta(n)$ $C_{worst}(n)=O(nm)$

模式匹配查找：从右向左；根据模式中是否有字符调整

① $S_0 \cdots \underset{\text{BARBER}}{S} \cdots S_{n-1}$ → BARBER 右移模式长度 m

② $S_0 \cdots \underset{\text{LEADER}}{MER} \cdots S_{n-1}$ → LEADER 左移模式长度 m

③ $S_0 \cdots \underset{\text{BARBER}}{B} \cdots S_{n-1}$ → BARBER 右移模式至最边的 C 和文本的 C 对齐

④ $S_0 \cdots \underset{\text{ORDER}}{AR} \cdots S_{n-1}$ → ORDER 右移至前 (m-1) 个字符中最左边的 C 和文本的 C 对齐

[ShiftTable] 字符 c　A　B　C　D　E … R …

例：BARBER 移动距离 t(c) 6-2=4　6-4=2　6　6　6-5=1　6　6-3=3　6

散列法　查找、插入、删除均为 $\Theta(1)$

(1) 开散列 ⇒ 分离链

例：key: A FOOL AND ARE SOON PARTED

address: 1 9 6 11 11 12

⇒ 0 1 2 3 4 5 6 7 8 9 10 11 12

A　AND　FOOL　ARE　PARTED　SOON

(2) 闭散列 ⇒ 开式寻址

当散列表接近满时，线性探查性能会恶化

0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND			FOOL		ARE	SOON
位置在后移	直接 store					直接 store			直接 store		直接 store	位置在后移

Warshall 算法　$\Theta(n^3)$

⇒ 求解有向图的传递闭包

通过一系列 n 阶矩阵构造传递闭包，$R^{(0)}$ 为邻接矩阵，$R^{(n)}$ 为传递闭包

从 $R^{(k-1)}$ 构造 $R^{(k)}$：新增了经过点 k 建立起的连接

$R^{(k-1)}$ = [j, k; k, i] ⇒ $R^{(k)}$ = [j, k; k, 1; i, 1, 1]　i和j通过k建立连接

Floyd 算法　$\Theta(n^3)$

⇒ 求解完全最短路径问题

通过一系列 n 阶矩阵来构造距离矩阵，$D^{(0)}$ 为权重矩阵，$D^{(n)}$ 为距离矩阵

从 $D^{(k-1)}$ 构造 $D^{(k)}$：更新了经过点 k 的两点间最短距离

$D^{(k-1)}$ = [j, k; k, d_{kj}; i, d_{ij}, d_{ik}] ⇒ $D^{(k)}$ = [j, k; k, d_{kj}; i, d_{ij}, d_{ik}]

i和j的最短距离经过点k后更新 $d_{ij}=\min\{d_{ij}, d_{ik}+d_{kj}\}$

最优二叉查找树

例：

key	A	B	C	D
P	0.1	0.2	0.4	0.3

主表：C[i,j] ⇒ 从i到j的最优二叉查找树的平均代价

C	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	1.4
3			0	0.4	1.0
4				0	0.3
5					0

$C(i,j)=\sum_{s=i}^{j} p_s + \min_{i\le k\le j}\{C(i,k-1)+C(k+1,j)\}$

∴ $C(1,4)=\min\{C(1,0)+C(2,4), C(1,1)+C(3,4), C(1,2)+C(4,4), C(1,3)+C(5,4)\}+1.0$

$=1.7$

根表：A[i,j] ⇒ k ⇒ 从i到j的最优二叉查找树的根是k

A	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					

C(1,0)+C(2,4) ⇒

C(1,1)+C(3,4) ⇒

C(1,2)+C(4,4) ⇒ ✓

C(1,3)+C(5,4) ⇒

动态规划的要素

① 最优化原理（最优子结构性质）⇒ 自底向上构造

② 无后向性 ⇒ 每个状态都是对过去历史的完整总结

③ 子问题的重叠性 ⇒ 动态规划的优势所在

Prim 算法 ⇒ 构造最小生成树

相连且权重最小

以一种贪婪的方式扩张当前生成树，将不在树中最近的顶点添加到小树中 ⇒ 不断扩张的子树

例：

① a(-,-)　b(a,3)　c(-,∞)　d(-,∞)　e(a,6)　f(a,5)

② b(a,3)　c(b,1)　d(-,∞)　e(a,6)　f(b,4)

③ c(b,1)　d(c,6)　e(a,6)　f(b,4)

④ f(b,4)　d(f,5)　e(f,2)

⑤ e(f,2)　d(f,5)　⑥ d(f,5)

权重矩阵效率：$\Theta(|V|^2)$　邻接链表效率：$O(|E|\log|V|)$

Kruskal 算法 ⇒ 构造最小生成树

按权值大小顺序访问每条边，若相应边加入子图后不产生回路，则把该边加入最小生成树边集中 ⇒ 扩张的子图

例：

bc 1 ① ✓　ef 2 ② ✓　ab 3 ③ ✓　bf 4 ④ ✓　cf 4 ⑤ ×　af 5 ⑥ ×　df 5 ⑦ ✓　ae 6　cd 6　de 8

到⑦时，边集已包含(n-1)个点，算法结束

Dijkstra 算法 ⇒ 求解加权图最短路径

由近及远，依次寻找离源点最近的顶点

例：

① a(-,-)　b(a,3)　c(-,∞)　d(a,7)　e(-,∞)

② b(a,3)　c(b,3+4)　d(b,3+2)　e(-,∞)

③ d(b,5)　c(b,7)　e(d,5+4)

④ c(b,7)　e(d,9)

⑤ e(d,9)

⇒ 适用于有向图和无向图，但权重必须非负

哈夫曼树及编码 ⇒ 变长编码

例：

0.1 B　0.15 _　0.2 C　0.2 D　0.35 A

⇒ 0.2 C　0.2 D　0.3　0.35 A　0.1 B　0.15 _ ⇒ 0.4　0.2 C　0.2 D　0.35 A　0.1 B　0.15 _ ⇒ 0.4　0.2 C　0.2 D

⇒ 0.4　0.6　0.2 C　0.2 D　0.35 A　0.1 B　0.15 _ ⇒ 1.0　0 0.4　1 0.6　0 0.2 C　1 0.2 D　0 0.35 A　1 0.1 B　0.15 _

A:11　B:100　C:00　D:01　_:101

线性规划问题：单纯形法

先在可行域中找到一个极点，然后检查能否在邻接极点取得更佳函数值，若不能，则该极点为最优点

① 初始化 ⇒ 转化成标准形式，建立单纯形表

max $z=3x+5y$　s.t. $x+y\le 4$, $x+3y\le 6$, $x,y\ge 0$

⇒ ① 最大化问题　max $z=3x+5y+0u+0v$　s.t. $x+y+u=4$, $x+3y+v=6$, $x,y,u,v\ge 0$　② 约束为等式且右端非负　③ 变量非负

基变量	所有变量 x	y	u	v	
u	1	1	1	0	4
v	1	3	0	1	6
目标行	-3	-5	0	0	0

→ x+y+u=4　→ x+3y+v=6　→ 3x+5y+0u+0v　当前目标函数值

① 最优测试 ⇒ 若目标行所有元素非负，则该表为一个最优解。

② 确定输入变量（进基变量）

目标行选取绝对值最大的负单元格为输入变量，所在列为主列

③ 确定分离变量（离基变量）若主元列的 $b\le 0$，则问题无界

对主列上每一个正单元格，其所在行最右单元格除以该单元格得到 θ 比率最小的行确定为分离变量、主元行

②③

	x	y	u	v	
u	1	1	1	0	4
换行 → v	1	3	0	1	6
目标行 →	-3	-5	0	0	0

主元列

$\theta_u=\frac{4}{1}=4$, $\theta_v=\frac{6}{3}=2$　min=2

④ 建立下一张表格　主元行变量替换为主元列变量

$row_1 = 1\cdot row_2'$

$row_2' = row_2 / 主元(3)$

$row_3 = (-5)\cdot row_2'$

	x	y	u	v	
u	2/3	0	2/3	0	2
y	1/3	1	0	1/3	2
	-4/3	0	0	5/3	10

⑤ 返回①，重复主测试成功　当前极点 (0,2,2,0)

最大流量问题：增益路径法 (Ford-Fulkerson 法)

1. 找出一个可行流

2. 标号以寻找增益路径
   - 前向边：i→j，有正的未使用容量 r_{ij}
   - 后向边：j→i，有正的流量 x_{ij}

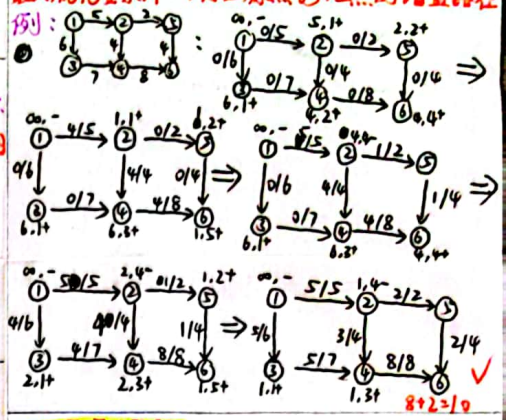
(1) 源点为 (∞, -)　V: 为前序顶点

(2) 一个端点 v_i 已标记且另一端点 v_j 未标记
- 前向边：v_j 标记 $(l_j, v_i, +)$，$l_j=\min\{l_i, r_{ij}\}$
- 后向边：v_j 标记 $(l_j, v_i, -)$，$l_j=\min\{l_i, x_{ij}\}$

(3) 重复(2)至汇点，得到标记，否则不存在增益路径

最大流充要条件：不存在源点到汇点的增益路径

例：



8+2=10 ✓

二分图最大匹配

找到增益路径：路径第1条边属于匹配M，第2条边不在M，以此类推，直到最后不属于M的边（长度为奇数）

当且仅当 M 不存在增益路径时，M 为最大匹配

稳定婚姻匹配：Gale-Shapley 算法

↳ 不存在受胁迫对（男士m和女士w未配对，但他们更倾向于对方）

九种情况：

① M1向W1求婚，W1接受

② M1向W1求婚，W1现有配对更优，W1拒绝

③ M1向W1求婚，优于W1现有配对，W1替换配偶

平凡下界

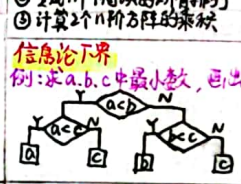
① 计算n次多项式　$\Omega(n)$　紧密

② 生成n个不同项的所有排列　$\Omega(n!)$　紧密

③ 计算2个n阶方阵的乘积　$\Omega(n^2)$　未知是否紧密

信息论下界

例：求a,b,c中最小数，画出决策树并求信息论下界



$h\ge\lceil\log_2 l\rceil$ 为信息论下界（l为叶子数，h为高度）

回溯法

例：n-皇后问题

每次只构造解的一个分量，并评估该部分构造解

⇒ 从部分解往前走，能进则进，不能则退回换一条路试

实现：状态空间树 ⇒ 生长方式：深度优先

生成组合对象

1. 生成排列
   ① 插入法　② Johnson-Trotter 法　③ 字典顺序法
2. 生成子集
   ① 减(-)治（特点：挤压序）　② 位串法　③ 格雷编码位串（类似长选择）

非比较型排序

1. 分布计数排序

统计元素出现的频次和分布值 ⇒ 基于分布反向填充目标数组

2. 桶排序

将元素根据划分的范围分布到"桶"中，然后对桶中元素PX

3. 基排序

将元素统一长度（缺位的补零），从最低位到最高位依次PX

B树 ⇒ 多路查找平衡树　$h\le\left\lfloor\log_{\lceil m/2\rceil}\frac{n+1}{2}\right\rfloor+1$

非叶子节点在键之间插入了指向子女节点的指针

背包问题动态规划求解

递推：$F(i,j)=\begin{cases}\max\{F(i-1,j), v_i+F(i-1,j-w_i)\}, & j-w_i\ge 0\\ F(i-1,j), & j-w_i<0\end{cases}$

→ 不包含物品i；　包含

初始条件：$F(0,j)=0$ and $F(i,0)=0$

例：$w_1=2, v_1=12; w_2=1, v_2=10; w_3=3, v_3=20; w_4=2, v_4=15$

包含物品 \ 容量	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

F(2,3) > F(1,3) ⇒ 包含物品2

⇒ 包含物品1

F(4,5) > F(3,5) ⇒ 包含

F(2,3) = F(3,3) ⇒ 不包含

背包问题分支界限法

根据是否包含物品i，列二叉树找最优解

扫描全能王 创建